Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

Parallel Programming
Assignment 2: Introduction to Multi-threading
Spring Semester 2026

Assigned on: **25.02.2026**

Due by: **(Wednesday Exercise) 02.03.2026**
(Friday Exercise) 04.03.2026

Overview

This week's assignment is about simple multi-threaded programs in Java. If you have any issues, please refer to the troubleshooting section at the end of the document.

- Fetch the new assignment from Gitlab by running:

  ```
  git fetch upstream
  git merge upstream/main
  ```

  You should now see the new assignment appear in your local directory.
- Open the project in IntelliJ IDEA: Click on *File* in the top-menu, then select *Open*. Select the folder of the new assignment.

Task 1 – Loop Parallelization

For this week's programming exercise you should learn how to improve the performance of executing a loop. For this purpose we will use threads to parallelize its execution.

Description: Our goal in this exercise will be to parallelize the execution of the following loop defined in `computePrimeFactors` method:

```
for (int i = 0; i < values.length; i++) {
  factors[i] = numPrimeFactors(values[i]);
}
```

which computes the number of prime factors for each element in an given array. For example, for number `12` the number of prime factors is `numPrimeFactors(12) = 3` since `12 = 2*2*3`. The implementation of `numPrimeFactors` is already provided for you in the assignment template and should not be changed.

Notice: Java libraries not included in the assignment project are not allowed. Do not rename any of the provided methods in the assignment (you can create additional classes and methods).

Tasks

A) To start with, print to the console `"Hello Thread!"` from a new thread. How do you check that the statement was indeed printed from a thread that is different to the main tread of your application? Furthermore, ensure that your program (i.e., the execution of main thread) finishes only after the thread execution finishes.

B) Run the method `computePrimeFactors` in a single thread other than the main thread. Measure the execution time of sequential execution (on the main thread) and execution using a single thread. Is there any noticeable difference?

C) Design and run an experiment that would measure the overhead of creating and executing a thread. That is, the time it takes to create a thread object with empty runnable, calling the `start` method and waiting for the thread to finish execution.

D) Before you parallelize the loop in task E), design how the work should be split between the threads by implementing method `PartitionData`. Each thread should process roughly equal amount of elements. Briefly describe your solution and discuss alternative ways to split the work.

E) Parallelize the loop execution in `computePrimeFactors` using a configurable number of threads.

F) Think of how would a plot that shows the execution speed-up of your implementation, for $n = 1$, 2, 4, 8, 16, 32, 64, 128 threads and the input array size of $100, 1000, 10000, 100000$ look like.

G) Measure the execution time of your parallel implementation for $n = 1$, 2, 4, 8, 16, 32, 64, 128 threads and the input array size of $100, 1000, 10000, 100000$. Discuss the differences in the two plots from task F) and G).

To understand your results, it may be a good idea to find out how many cores your system has available. In Java, you can query the Runtime class to find this information:

```
int cores = Runtime.getRuntime().availableProcessors();
```

Creating Threads in Java

Java offers a couple of possibilities to create new threads. Below we briefly cover the most common options. Regardless of the variant used to create the Thread object, we always call the `start()` method on thread objects to actually run the associated threads.

The most compact option is to create the code that is run by a thread anonymously and inline:

```
// create thread
Thread t = new Thread(){
  public void run(){
    // code to execute in the thread
  }
};
// start thread
t.start();
```

The second – and most flexible option – is to separate the task you want to accomplish and the execution of said task. This can come in handy if you have a computation that relies on a lot of shared state but is at the same time fairly complicated. In that scenario you want the class that implements your task to implement `Runnable`. Using this method you can share a single task object between multiple threads:

```
public class MyOperation implements Runnable {
  private final int initialValue;

  public MyOperation(int initialValue) {
    this.initialValue = initialValue;
  }

  public void run() {
    // code to execute when run
  }

  // can define more methods and fields here
}
```

Now we can create threads using a `MyOperation` object:

```
int initVal = 5;
MyOperation op = new MyOperation(initVal);
Thread t = new Thread(op);
t.start();
```

It is also possible to create a custom class that inherits from Thread:

```
public class MyThread extends Thread {
  // thread local data
  private final int initialValue;

  public MyThread(int initialValue) {
    this.initialValue = initialValue;
  }

  public void run(){
    // code to execute in thread
  }

  // can define more methods and fields here
}
```

We can then create a new thread using a new instance of our custom class:

```
// create thread
int initValue = 5;
Thread t = new MyThread(initValue);
// start thread
t.start();
```

Measuring Time

In Java you can measure time in nano-second increments using the snippet listed below.

```
long startTime = System.nanoTime();

// the code you want to measure time for goes here

long endTime = System.nanoTime();
long elapsedNs = endTime - startTime;
double elapsedMs = elapsedNs / 1.0e6;
```

Submission

You need to submit your code and reports by using our submission system. This step will be the same for all of the subsequent exercises, but we will explain it in detail here so you can familiarize yourself with the system.

To submit your code using the terminal, follow the instructions below:

- In the terminal, navigate to your personal repository.
- Add your changes:
  ```
  git add .  # this command adds all changes in the git repo
  ```
- Commit your changes. In your commit message, make sure to include the [Submission] tag.
  ```
  git commit -m "[Submission] ..."
  ```
- Push your changes!
  ```
  git push
  ```

To submit your code using IntelliJ, follow the instructions below:

- Press the "Commit" button (found on the left hand toolbar)
- Select all the files you want changed during the assignment.
- Write a descriptive commit message. Make sure to include the [Submission] tag.
- Press "Commit and Push"

After this step, you can check your repository on the GitLab website in your browser. Make sure that you see your changes there!

Troubleshooting

Git Issues
Missing repo:
In rare cases, you might get an error like this:

```
Can't connect to any URI
https://gitlab.inf.ethz.ch/COURSE-PPROG26/pprog26-<username >.git
(https://gitlab.inf.ethz.ch/COURSE-PPROG26/pprog26-<username>.git:
https://gitlab.inf.ethz.ch/COURSE-PPROG26/pprog26-<username >.git/info/refs?
service=git-receive-pack not found)
```

It means that no repository was created for your username yet. If that is the case, please contact your teaching assistant. The teaching assistant will be able to create the repository location for you. After that, you should be able to submit your work by following the instructions above.

Issues fetching from upstream:
If you are unable to fetch upstream, or get issues related to SSH vs HTTPS when merging from upstream, you can change your upstream URL using the following command:

```
git remote set-url upstream https://gitlab.inf.ethz.ch/course-pprog26/pprog-26-assignments.git
```

IntelliJ / Java Issues
If you have trouble running the Java programs in IntelliJ, check the following:

- You have marked the `src` directory as a “Source Root”. Right click on the folder `src`, select “Mark Directory As >Source Root”.
- You have installed an updated JDK. We recommend `openjdk-25`. You can check this under “File >Project Structure >SDK”